

Chapter 6.

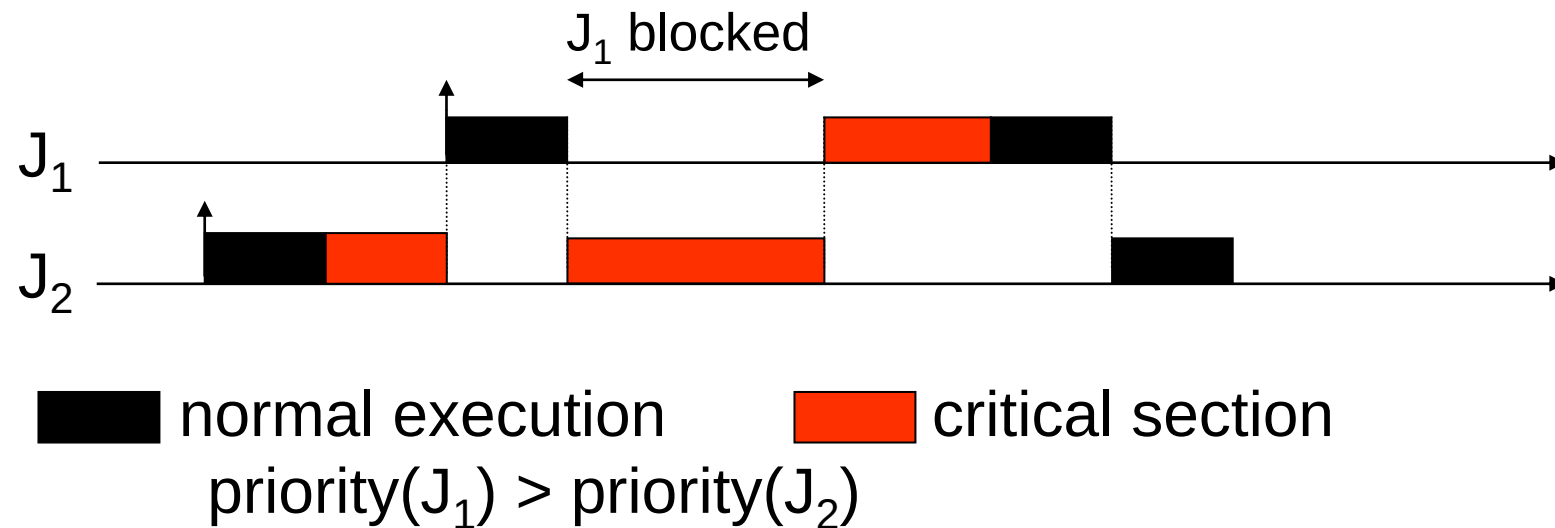
Scheduling with resource constraints

Structure of this course

- Chapter 1: Brief review of general-purpose Operating Systems
- Chapter 2: Introduction into Embedded Real-Time Operating Systems
 - Basic notions, basic concepts, basic challenges
- Chapters 3 – 6: Scheduling and strict schedulability analysis as a **main component of safety analysis** for Embedded Real-Time Operating Systems
 - **Chapter 6: Scheduling with resource constraints**
- Chapter 7: Worst-Case Execution Time Analysis based on **Formal Methods** as a precondition for precise schedulability analysis
- If time allows:
 - (Chapter 8: **Security** of Embedded Operating Systems)
 - (Chapter 9: **Formal Verification of Safety and Security** of Embedded Operating Systems)

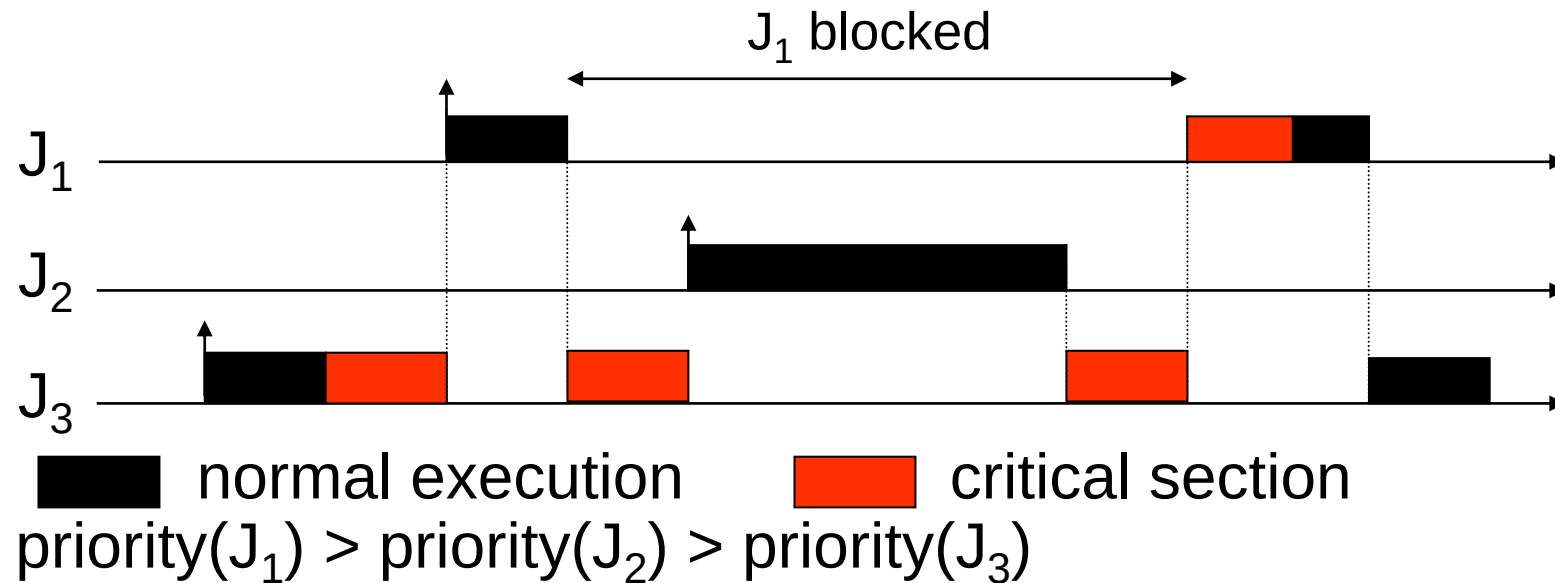
Scheduling with resource constraints – The priority inversion problem

- **Priority inversion** can occur due to resource conflicts (exclusive use of shared resources) in fixed priority schedulers like RM:



- Blocking time equal to length of critical section.

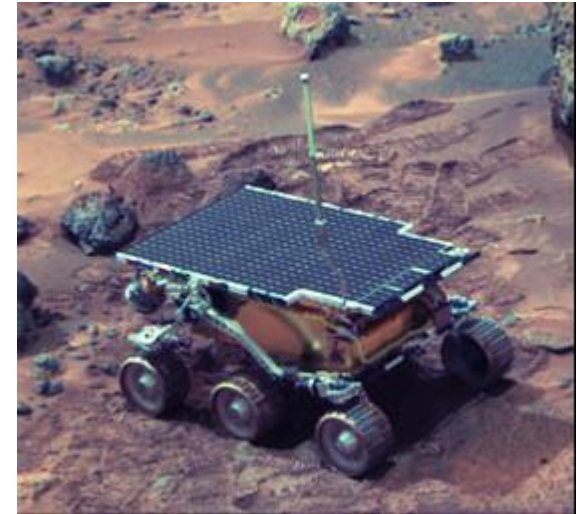
The priority inversion problem



- Blocking time equal to length of critical section + computation time of J_2 .
- **Unbounded time of priority inversion**, if J_3 is interrupted by tasks with priority between J_1 and J_3 during its critical section.

Priority inversion in real life: The MARS Pathfinder problem (1)

“But a few days into the mission, not long after Pathfinder started gathering meteorological data, the spacecraft began experiencing total system resets, each resulting in losses of data. The press reported these failures in terms such as "software glitches" and "the computer was trying to do too many things at once".” ...



© <https://photojournal.jpl.nasa.gov/catalog/PIA01122>

Priority inversion in real life: The MARS Pathfinder problem (2)

- System overview:
 - **Information Bus (IB):**
 - Buffer for exchanging data between different tasks
 - Shared resource of two tasks M and B
 - Three tasks:
 - **Meteorological data gathering task (M):**
 - collects meteorological data
 - reserves IB, writes data to IB, releases IB
 - infrequent task, low priority in RM scheduler
 - **Bus management (B):**
 - data transport from IB to destination
 - reserves IB, data transport, releases IB
 - frequent task, high priority

Priority inversion in real life: The MARS Pathfinder problem (3)

- Three tasks:
 - ...
 - “**Communication task**” (C):
 - medium priority, does not use IB
- Scheduling with fixed priorities.
- **Watch dog timer (W):**
 - Execution of B as indicator of system hang-up
 - If B is not activated for certain amount of time: Reset the system

Priority inversion in real life: The MARS Pathfinder problem (4)

(see http://research.microsoft.com/~mbj/Mars_Pathfinder/)

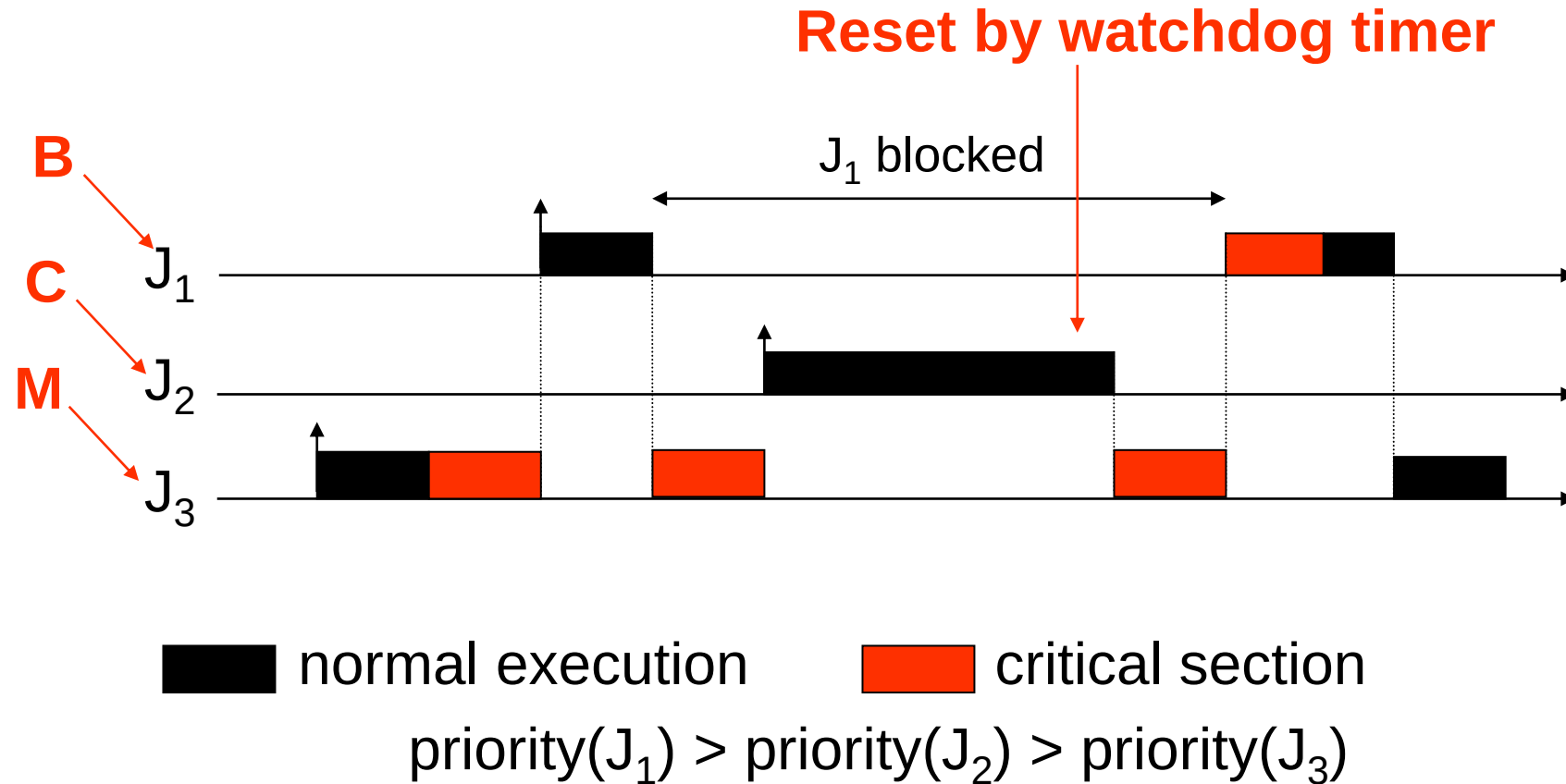
“Most of the time this combination worked fine.

However, very infrequently it was possible for an interrupt to occur that caused the (medium priority) communication task to be scheduled during the short interval while the (high priority) information bus thread was blocked waiting for the (low priority) meteorological data thread. In this case, the long-running communications task, having higher priority than the meteorological task, would prevent it from running, consequently preventing the blocked information bus task from running.

After some time had passed, a watchdog timer would go off, notice that the data bus task had not been executed for some time, conclude that something had gone drastically wrong, and initiate a total system reset.

This scenario is a classic case of priority inversion.”

Priority inversion in real life: The MARS Pathfinder problem (5)



Coping with priority inversion: Simple solution

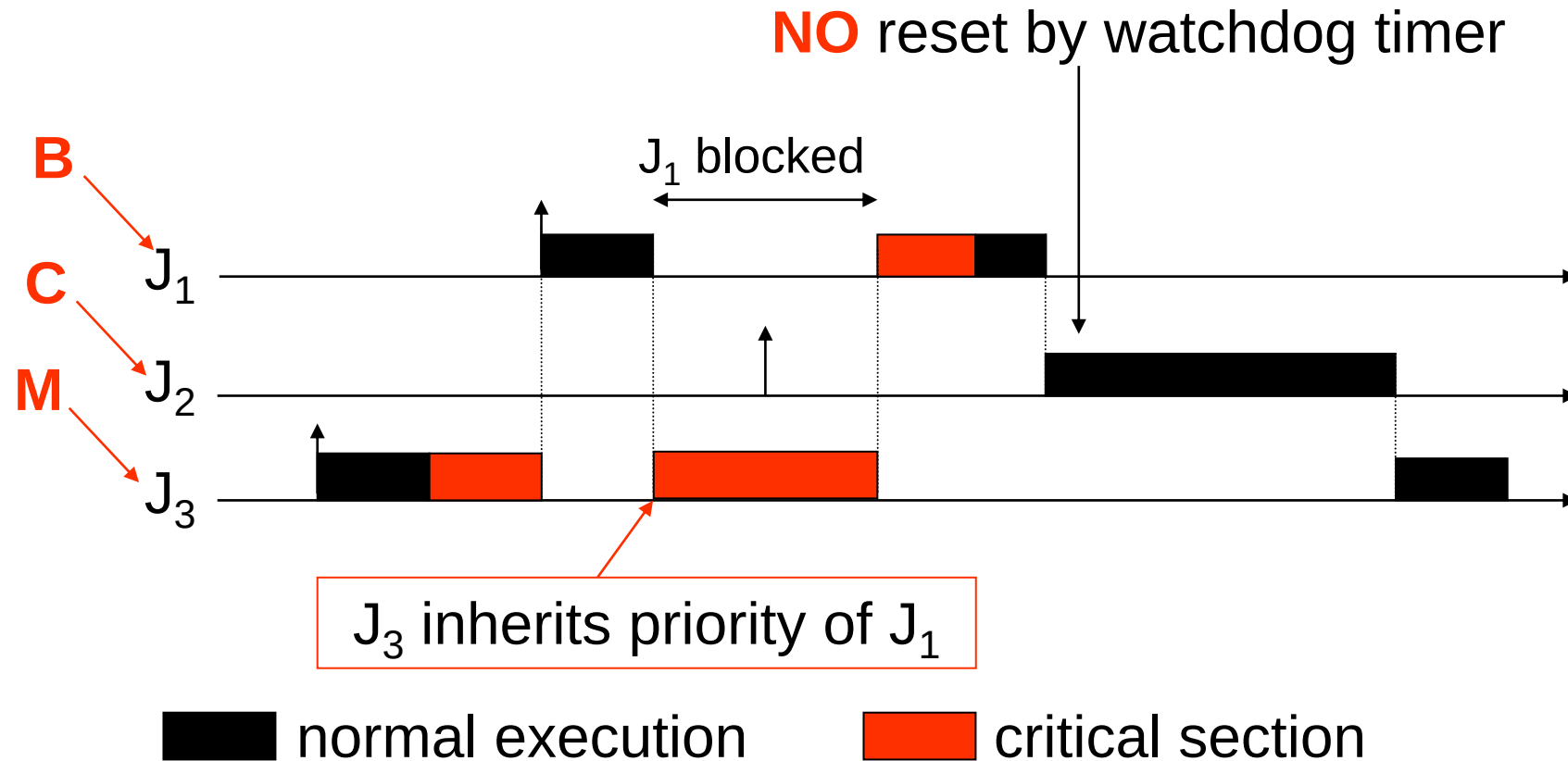
- Disallow preemption during execution of critical sections
- Disadvantages:
 - Unnecessary blocking possible (even for critical tasks which never enter critical sections)
 - Not suitable for long critical sections

Coping with priority inversion: The priority inheritance protocol

Idea of **priority inheritance protocol**:

- If a task J_h blocks, since another task J_l with lower priority owns the requested resource, then J_l inherits the priority of J_h .
- When J_l releases the resource, the priority inheritance from J_h is undone.
- Rule: Tasks always inherit the highest priority of tasks blocked by it.

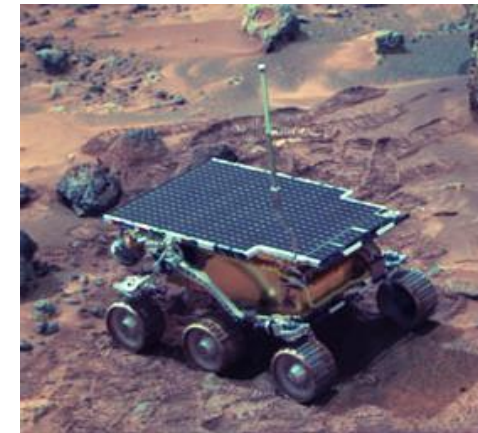
Priority inheritance for the Pathfinder example



$\text{priority}(J_1) > \text{priority}(J_2) > \text{priority}(J_3)$

Priority inversion on Mars

- Priority inheritance also solved the Mars Pathfinder problem:
 - the VxWorks operating system used in the pathfinder implements a flag for the calls to mutex primitives.
 - This flag allows priority inheritance to be set to “on”.
 - When the software was shipped, it was set to “off”.



© <https://photojournal.jpl.nasa.gov/catalog/PIA01122>

- **The problem on Mars was corrected by using the debugging facilities of VxWorks to change the flag to “on”, while the Pathfinder was already on the Mars [Jones, 1997].**

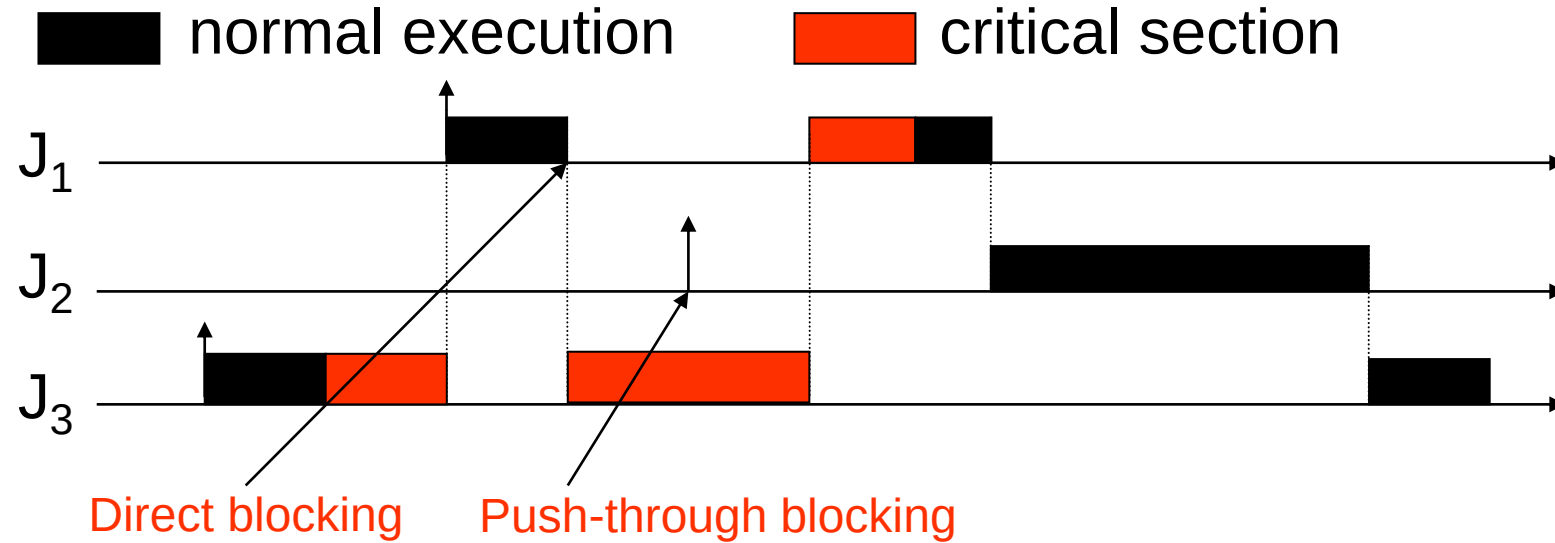
Properties of priority inheritance and schedulability analysis

- Assume n tasks $J_1, \dots, J_n$.
- Task J_i has
 - a fixed **nominal** priority P_i
 - an dynamic **active** priority p_i with $p_i \geq P_i$
- Initially $p_i := P_i$.
- Assume $P_1 > P_2 > \dots > P_n$
- There are m shared resources $R_1, \dots, R_m$ for which mutual exclusion is guaranteed by semaphores $S_1, \dots, S_m$
- It is possible to have several (possibly nested) critical regions of a single task.
- We assume that critical sections are
 - disjoint or
 - “properly nested”, i.e., if a task may hold several semaphores at the same time, then the releases are in reverse order of requests.

Exact protocol definition

- Jobs are scheduled based on their **active** priorities p_i . (Jobs with the same priority are executed in a First Come First Served discipline.)
- When a job J_i tries to **enter** a critical section R_c and is **blocked** on the semaphore S_c ,
 - it transmits its active priority to the job J_k that holds that semaphore. (“ J_k inherits active priority p_i .”)
 - Hence J_k resumes and starts to execute the rest of its critical section with priority $p_k = p_i$.
 - In general, a task inherits the highest priority of the jobs blocked by it.
- When J_k **leaves** a critical section R_c ,
 - it unlocks the semaphore S_c , so that the highest-priority job blocked on that semaphore is awakened (if there is such a job)
 - $p_k = P_k$, if no other jobs are blocked by J_k
 - $p_k =$ highest priority of the jobs blocked by J_k otherwise
- Priority inheritance is **transitive**, i.e., if J_3 blocks J_2 , J_2 blocks J_1 , then J_3 inherits the priority of J_1 via J_2 .

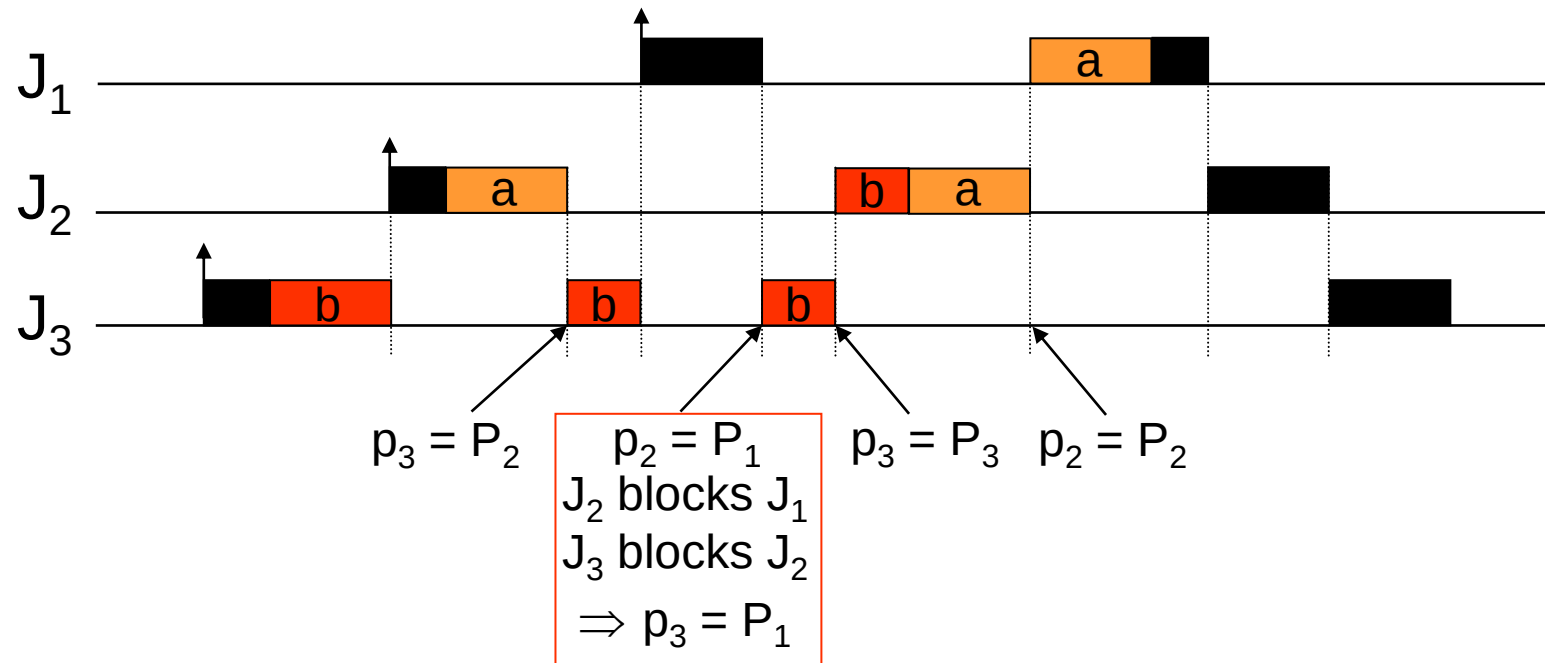
Two types of blocking



- Two kinds of blocking of a higher-priority job:
 - Direct blocking:** Higher-priority job tries to enter critical section held by lower-priority job.
 - Needed to ensure mutual exclusion.
 - Push-through blocking:** Medium-priority job blocked by lower-priority job that has inherited a higher priority from a job it blocks directly.
 - Needed to avoid unbounded priority inversion!

Example for transitive priority inheritance

- J_1 uses resource R_a , guarded by S_a
- J_3 uses resource R_b , guarded by S_b
- J_2 uses R_a and R_b .



Schedulability analysis

- **Important observation** for schedulability analysis:
 - Due to the priority inheritance protocol we can provide an estimate on the **maximum blocking time B_i** for some task J_i which is caused by tasks with lower priority.
 - B_i is based on the length of critical sections (which can be computed by WCET analysis).
- **Lemma:** If there are $n-i$ lower-priority tasks that can block J_i , then J_i can be blocked for at most the duration of $n-i$ critical sections (one for each of the $n-i$ lower priority jobs), regardless of the number of semaphores used by J_i .

Maximum blocking time

- **Proof:**

- J_i can be blocked by a lower-priority job J_k only if J_k is in a critical section R_j and has inherited a priority $\geq P_i$ (from J_i or from a task with higher priority – blocking of J_i by semaphore S_j is only one possibility for this situation).
- After J_k has left the “outermost critical section of R_j ” (which may be different from R_j in case of (properly) nested critical sections) we have the situation that J_k is not able to enter a new critical section before J_i has finished.
- This argument holds for all $n-i$ lower-priority tasks.

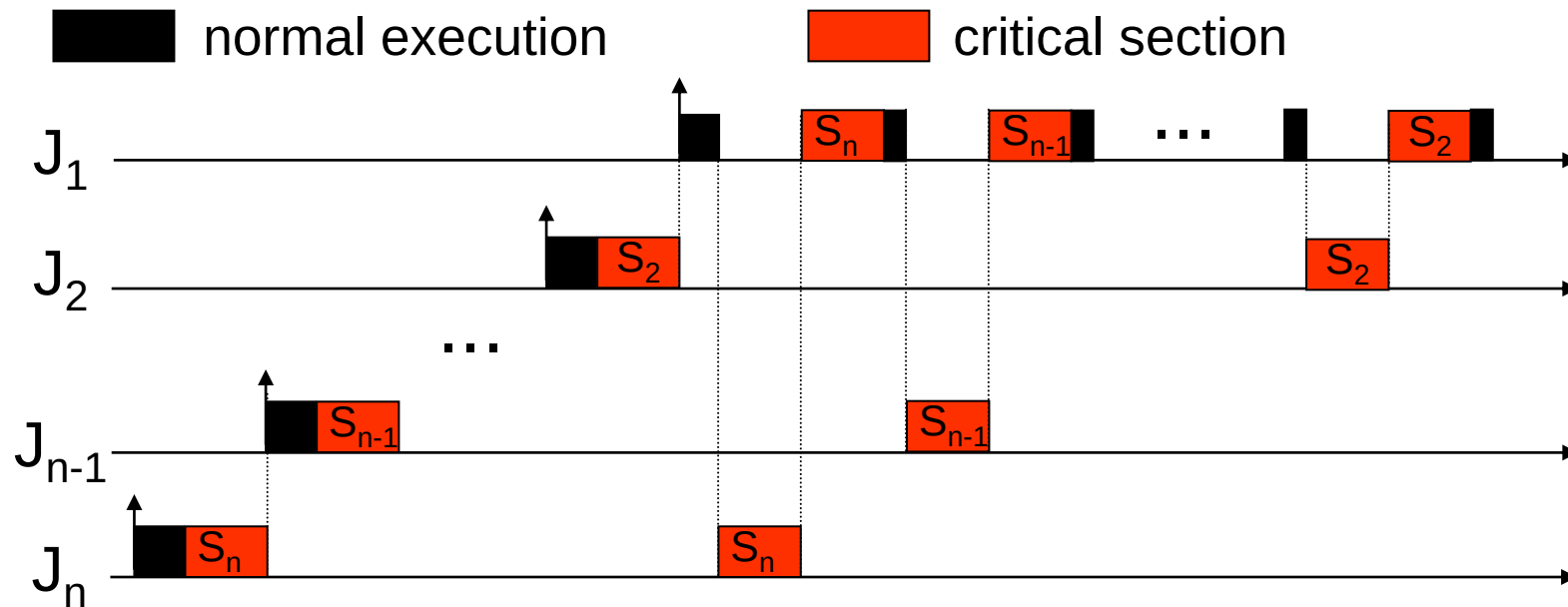
- **Corollary 1:** If $b_{\max}$ is the maximal length of a critical section, then the maximum blocking time of J_i is bounded by $B_i \leq (n-i) b_{\max}$.
- (More refined analysis possible.)
- **Corollary 2:** Even the **sum** B_i^{sum} of all blocking times for $J_1, \dots, J_i$ caused by $n-i$ lower-priority tasks during the interval $[a_i, f_i]$ is bounded by the duration of $n-i$ critical sections.

Schedulability analysis

- Here for n **periodic** tasks $\tau_1, \dots, \tau_n$.
- τ_i has period T_i and computation time C_i .
- Let us assume a fixed priority scheduler, e.g. RM.
- Simple solution: Schedulability checks can be adjusted by adding B_i^{sum} to C_i .
- Example:
 - Assume $T_1 < T_2 < \dots < T_n$.
 - Change recurrent equation for response times
 - $R_i = C_i + \sum_{k=1}^{i-1} \lceil R_i/T_k \rceil C_k$ into
 - $R_i = C_i + B_i^{\text{sum}} + \sum_{k=1}^{i-1} \lceil R_i/T_k \rceil C_k$.

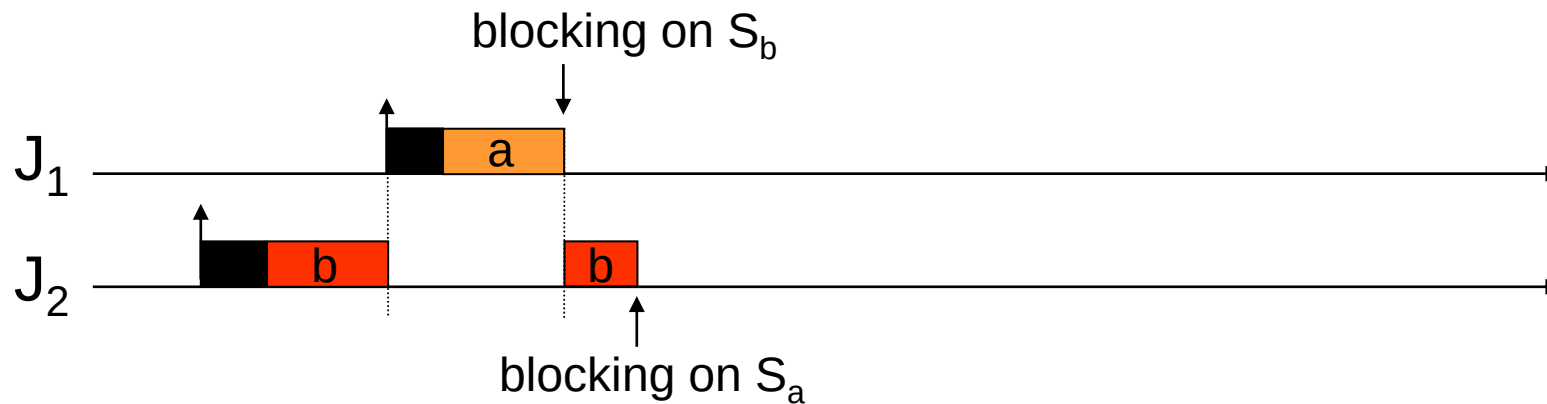
Problems with priority inheritance

- If a high priority process needs different resources, which are owned by other processes, it has to wait several times during its execution (“**chained blocking**”).
- Example:
 - J_1 uses semaphores $S_2, \dots, S_n$.
 - J_2 uses semaphore $S_2, \dots, J_n$ uses semaphore S_n .



Problems with priority inheritance

- Deadlocks are possible due to improper use of semaphores.
 - Example: J_1 requests resource R_a , then resource R_b . J_2 requests R_b , then R_a .



Problems with priority inheritance

- Chained blocking, deadlocks
- **Solution:**
More sophisticated protocol, “**priority ceiling protocol**”.
 - Prevents chained blocking and deadlocks.

Solution: Priority ceiling protocol

- Prevents chained blocking and deadlocks.
- **Idea:**
 - Add a new rule to priority inheritance which may block tasks upon a request for a **free** semaphore (“**ceiling blocking**”).
 - This rule ensures that there are never two or more tasks holding semaphores that are able to block the same higher-priority task.
- Ceiling blocking is based on the definition of a **priority ceiling $C(S_k)$** for each semaphore S_k :
$$C(S_k) = \max \{P_i \mid \text{in } J_i \text{ there is a request to semaphore } S_k\}$$
- Priority ceiling $C(S_k)$ is a static value, can be computed off-line.

Protocol definition: Priority ceiling as extension of priority inheritance (1)

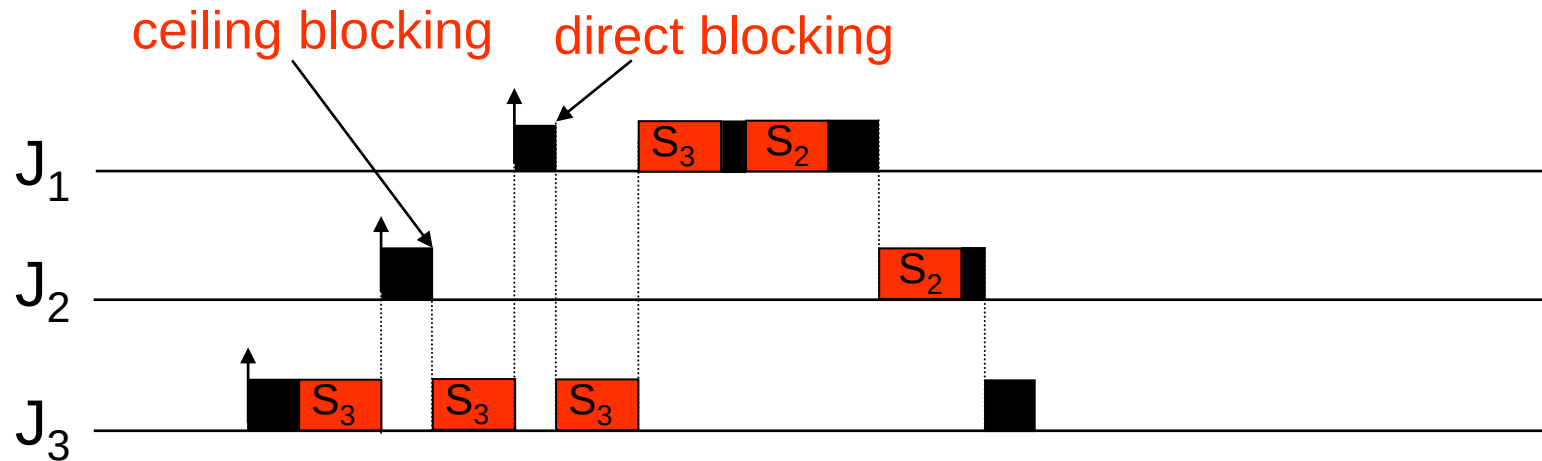
- Jobs are scheduled based on their active priorities p_i . (Jobs with the same priority are executed in a First Come First Served discipline.)
- When a task J_i tries to enter a critical section guarded by semaphore S_k , then the following algorithm decides whether J_i is allowed to enter:
 - Consider all semaphores currently locked by jobs other than J_i , let S^* be the semaphore with highest priority ceiling among them, let $C(S^*)$ be its ceiling
 - If $P_i > C(S^*)$, then J_i enters the critical region.
 - If $P_i \leq C(S^*)$, then J_i blocks; J_i is said to be blocked on semaphore S^* by the job that holds the lock on S^* .
- ...

Protocol definition: Priority ceiling as extension of priority inheritance (2)

- ...
- When a job J_i tries to **enter** a critical section R_c and is **blocked** on the semaphore S_c ,
 - it transmits its ~~active~~ priority to the job J_k that holds that semaphore. (“ J_k inherits priority P_i .”)
 - Hence J_k resumes and starts to execute the rest of its critical section with priority $p_k = P_i$.
 - ~~In general, a task inherits the highest priority of the jobs blocked by it.~~
- When J_k **leaves** a critical section R_c ,
 - it unlocks the semaphore S_c , so that the highest-priority job blocked on that semaphore is awakened (if there is such a job)
 - $p_k = P_k$, if no other jobs are blocked by J_k
 - $p_k =$ highest priority of the jobs blocked by J_k otherwise
- ~~Priority inheritance is **transitive**, i.e., if J_3 blocks J_2 , J_2 blocks J_1 , then J_3 inherits the priority of J_1 via J_2 .~~

Priority ceiling: Example

- Example:
 - J_1 uses semaphores S_2 and S_3 .
 - J_2 uses semaphore S_2 , J_3 uses semaphore S_3 .
 - $C(S_2) = C(S_3) = P_1$.



Properties of priority ceiling

- **Theorem (*):** Under the Priority Ceiling Protocol, a job J_i can be blocked for at most the duration of one critical section.
- **Proof:**
 - Let t_0 be the arrival time of J_i .
 - Assume that there are two different lower-priority tasks J_{k_1} and J_{k_2} which are executed during the execution interval of J_i .
 - Let $t_1 > t_0$ be the first time of execution of J_{k_1} ,
 $t_2 > t_0$ be the first time of execution of J_{k_2} .
 - At time t_1 J_{k_1} inherits a priority $P_{l_1} \geq P_i$.
 - I.e., J_{l_1} tries to enter critical section R_c at t_1 , but ceiling blocking „by J_{k_1} “ occurs $\Rightarrow J_{k_1}$ holds a semaphore S_a with $C(S_a) \geq P_{l_1}$.
 - J_{k_1} holds a semaphore S_a with $C(S_a) \geq P_{l_1} \geq P_i$ at time t_1 and thus at time t_0 .

Properties of priority ceiling

■ Proof:

- ...
- At time t_2 J_{k_2} inherits a priority $P_{l_2} \geq P_i$.
 - With same arguments: J_{k_2} holds a semaphore S_b with $C(S_b) \geq P_i$ at time t_2 and thus at time t_0 .
- Assume that J_{k_1} entered R_a at time $t_a < t_0$, J_{k_2} entered R_b at time $t_b < t_0$.
- Case $t_a < t_b$:
 - At time t_b : Let S^* be the semaphore with highest priority ceiling among all semaphores locked by jobs other than J_{k_2} .
 - $C(S^*) \geq C(S_a) \geq P_i > P_{k_2}$
 $\Rightarrow J_{k_2}$ can not enter R_b at time t_b due to ceiling blocking! Contradiction!
- Case $t_a > t_b$: Analogously.

$\Rightarrow$ At most one lower-priority task J_k may be executed during the execution interval of J_i .

- If R_a is the outermost critical section that J_k holds at time t_0 , then J_k can not block J_i any more after J_k has left R_a . $\Rightarrow J_i$ is blocked for at most the duration of one critical section.

Properties of priority ceiling (1)

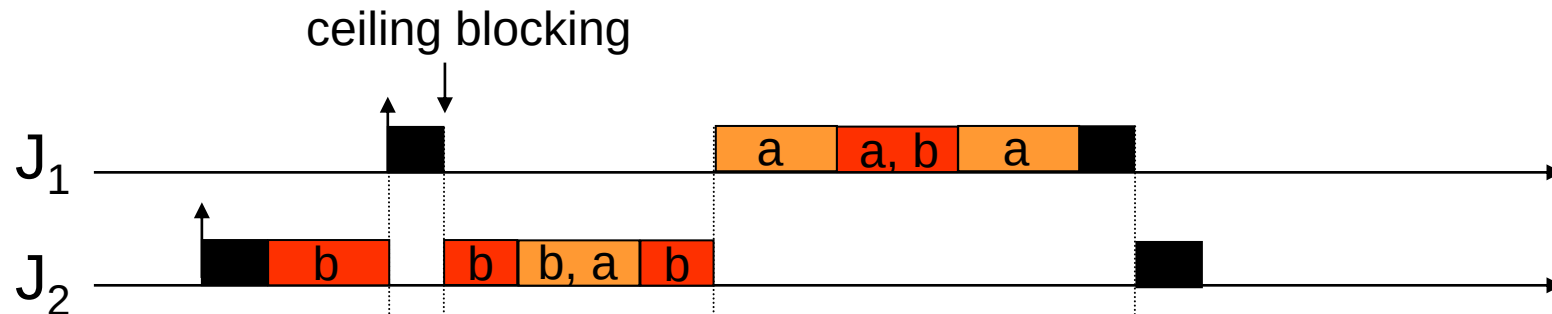
- If $b_{\max}$ is the maximal length of a critical section, then the maximum blocking time B_i of a task J_i is bounded by $b_{\max}$!
- Refined analysis:
 - Which tasks can **really** block J_i ?
 - $\text{cand}_i = \{J_k \mid P_k < P_i, \text{ there is a request in } J_k \text{ to semaphore } S_a \text{ with } C(S_a) \geq P_i\}$
 - $\Rightarrow$ Refined bound on blocking time b_i of J_i :
 $b_i = \max\{\text{length}(R_a) \mid \text{there is a request to semaphore } S_a \text{ with } C(S_a) \geq P_i \text{ in } J_k \text{ with } P_k < P_i\}$

Response time analysis

- **Corollary:** Let a_i be the arrival time of task J_i , f_i its finishing time. Under the Priority Ceiling Protocol, all tasks with priority $\geq P_i$ can be blocked during $[a_i, f_i]$ for at most the duration of one critical section.
- (Follows from the proof of Theorem (*).)
- Consider $J_1, \dots, J_n$ with $P_1 > \dots > P_n$.
- Change recurrent equation for RM response analysis
 - $R_i = C_i + \sum_{k=1}^{i-1} [R_i/T_k] C_k$ into
 - $R_i = C_i + b_i + \sum_{k=1}^{i-1} [R_i/T_k] C_k$.

Properties of priority ceiling (2)

- **Theorem:** Deadlocks are not possible with Priority Ceiling Protocol.
- Example:
 - J_1 requests resource R_a , then resource R_b . J_2 requests R_b , then R_a .
 - $C(S_a) = C(S_b) = P_1$.



Proof: Deadlocks are not possible with PCP

- **Ressource allocation graph:**

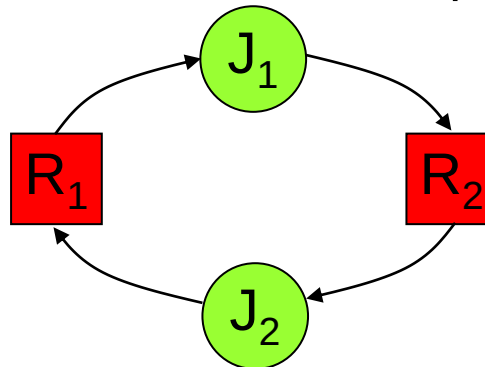
- Two types of nodes:

- Tasks J_i
 - Ressources (critical regions) R_k

- Edges:

- $R_k \rightarrow J_i$: R_k belongs to J_i (J_i is in c.r. R_k)
 - $J_i \rightarrow R_k$: J_i wants R_k , but did not get access to it

- Deadlocks due to ressource conflicts correspond to cycles in the ressource allocation graph:



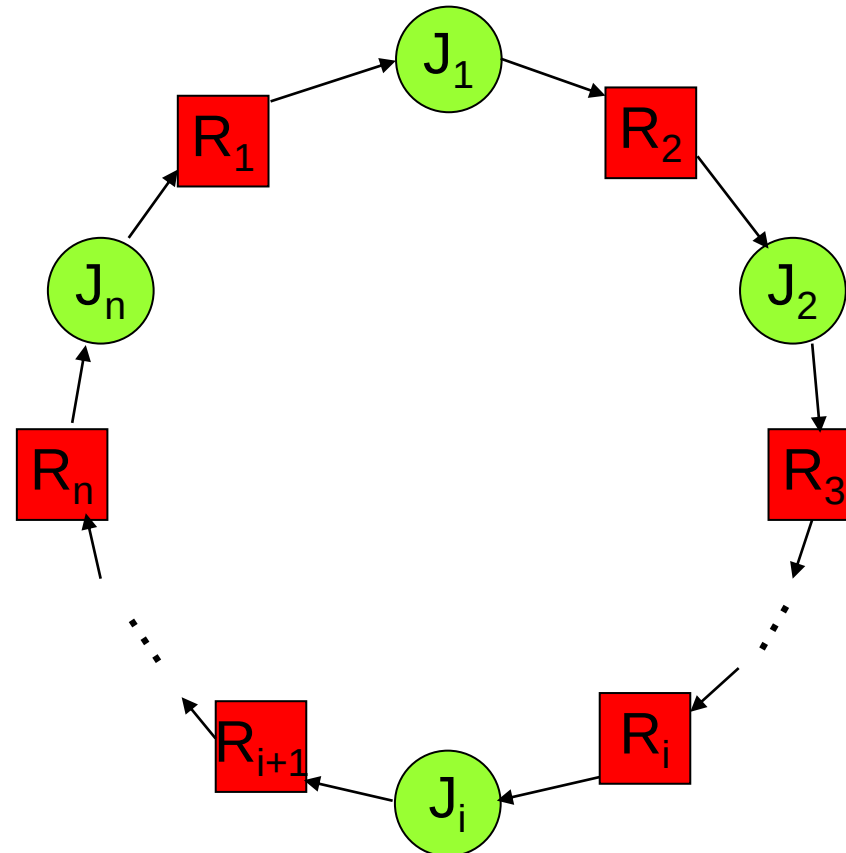
Proof: Deadlocks are not possible with PCP

- **Lemma 1:** Consider J_i, J_k with $P_i > P_k$. Under priority ceiling protocol: If J_i is in a critical section R_a , then J_k will not be executed before the finishing time f_i of J_i .
- Proof:
 - Assume J_i enters a critical section at time t , then $P_i > C(S^*)$ where S^* is the semaphore with highest priority ceiling among all semaphores currently locked by jobs other than J_i .
 - Thus, $P_i > C(S_b)$ where S_b is any semaphore locked by J_k at time t (*).
 - Before f_i , J_k could only be executed, if it would inherit a priority $\geq P_i$.
 - J_k can only inherit a priority $\geq P_i$, if some task J_l with priority $P_l \geq P_i$ would block on a semaphore S_b locked by J_k with $P_l \leq C(S_b)$. This is not possible, since $P_l \geq P_i > C(S_b)$ (see (*)).

Proof: Deadlocks are not possible with PCP

- **Proof:**

- Assume that there is a deadlock at time t_3 .
- W.l.o.g. the resource allocation graph at time t_3 contains the following cycle:



Proof: Deadlocks are not possible with PCP

- **Proof:**

- ...
- Let J_i be an arbitrary task under $J_1, \dots, J_n$.
- Assume that J_i successfully allocated resource R_i at time t_1 , **only tried** to allocate R_{i+1} at time t_2 .
 $\Rightarrow t_1 < t_2 \leq t_3$

Proof: Deadlocks are not possible with PCP

- **Proof:**

- ...

- **Case 1:** At time t_2 there is a task J with priority $> P_i$ owning some resource R .
 - Contradiction. Due to Lemma 1, task J_i cannot try to get R_{i+1} at time t_2 , since it will not be executed before the finishing time of J .
 - **Case 2:** At time t_2 there is no task with priority $> P_i$ owning any resource.
 - Assume that at time t_2 J_i is blocked on semaphore S^* currently locked by a task J_l .
 $\Rightarrow P_l < P_i \leq C(S^*)$ (**)
 - Assume that task J_l obtained the lock on S^* at time t_0 .
 - $t_0 > t_1$ is not possible due to Lemma 1 (J_i with higher priority than J_l obtained the lock on R_i at time $t_1 < t_0$.)

Proof: Deadlocks are not possible with PCP

■ Proof:

- ...
- Thus $t_0 < t_1$.
- At time t_1 the „highest priority ceiling among all semaphores currently locked by jobs other than J_i ” is $C \geq C(S^*)$ (***) (since J_i has the lock on S^* since t_0 and at time t_1).
- Contradiction to „ J_i successfully allocated resource R_i at time t_1 “, since $P_i \leq C(S^*) \leq C$ (due to (**) and (***)).
 J_i would have blocked at time t_1 .

- **Corollary:** Under the priority ceiling protocol it cannot happen that a task which currently locks a semaphore is blocked due to another semaphore.